

Week 5

Basic Data Analytics: Exploratory Data Analysis

- Feature Engineering
- Visualization

Features Engineering

- Feature engineering is the process of transforming selected features in a dataset to create certain patterns, provide insight, and improve understanding of the data.
- This will eventually improve the accuracy of the model when trained with the data.
- **Key purposes:**
 - Improve data quality
 - Enhance model accuracy
 - Simplify interpretation
 - Prepare for visualization and modeling

Features Engineering

1. **Enhanced model performance with well-engineered features:** When feature engineering techniques are carried out on features in a dataset, machine learning models are provided with reliable data that enables them to provide better accuracy and results.
2. **Improved data representation and pattern extraction:** Properly engineered or transformed features provide reliable and detailed insights into data. This also aids data scientists or analysts in drawing out valuable conclusions from it.
3. **Dimensionality reduction and prevention of overfitting:** Dimensionality reduction involves removing or filtering unuseful or irrelevant features which in turn will yield better model performance, especially in high dimension data. Dimensionality reduction reduces the chance of model overfitting.

Features Engineering

4. **Handling missing data effectively:** Feature engineering involves methods in which missing data are handled without harming model performance.
5. **Incorporating domain knowledge into the model:** Applying feature engineering techniques allows us to include domain knowledge by selecting useful features and removing irrelevant features in the dataset before training in the machine learning model.

Feature Engineering Techniques

- Handling missing data
 - i. **Mean/Median/Mode imputation:** This is taking the mean, median, or mode of all other values in the feature and inputting the derived value into empty or missing columns.
 - ii. **Forward Fill/Backward Fill:** The forward fill / backward fill method process of filling in the missing values with the previous known value or the next value in front of the missing value.
 - iii. **Interpolation:** This is the process of estimating and filling in missing values using techniques like linear or polynomial interpolation.
 - iv. **K-nearest neighbors imputation:** Fill in missing data using the features derived from using the KNN(K- Nearest Neighbors) algorithm on the other features of the dataset.

Feature Engineering Techniques

- Handling categorical data
 - i. **One-hot encoding:** The categorical variables are converted or transformed into binary(0 and 1) vectors assigned as a separate feature to the dataset..
 - ii. **Label encoding:** The label encoding technique assigns a respective integer value to each categorical variable.
 - iii. **Target encoding:** This encoding scheme assigns the mean or median of the target variable to each category.

Feature Engineering Techniques

- Feature scaling
 - i. **Min-Max Scaling (Normalization):** Min-max is a feature scaling technique that normalizes features in a dataset between a minimum and maximum value.
 - ii. **Standardization:** Standardization converts the features in a dataset using the mean and standard deviation. The value of the mean is 0 and the standard deviation is 1.

Feature Engineering Techniques

- Feature selection
 - i. **Univariate feature selection:** Univariate feature selection removes all features whose variance doesn't meet a particular threshold value.
 - ii. **L1 regularization (Lasso):** Lasso regression algorithm can reduce the coefficients and remove those with lower values.

Feature Engineering Workflow

- i. Understand the context and data meaning
- ii. Handle missing and outlier values
- iii. Encode categorical variables
- iv. Transform and scale features
- v. Create derived features
- vi. Validate using correlation or domain knowledge

Exercise

- Use the same dataset (power_consumption.csv) and apply:
- Fill missing values
- Cap outliers
- Create $\text{efficiency} = \text{power_kW} / \text{vibration_level}$
- Encode status
- Normalize numeric columns

Visualization

- Key Visualization Goals
 - ✓ Summarize data intuitively
 - ✓ Detect patterns and outliers
 - ✓ Compare groups or features
 - ✓ Validate assumptions visually
- Python Libraries for Visualization
 - i. Matplotlib → Foundation library for plotting
 - ii. Seaborn → High-level interface for statistical visualization
 - iii. Plotly → Interactive visualization (more suitable for BI)

Matplotlib

- Matplotlib is a plotting library for the Python programming language and its numerical mathematics extension NumPy.
 - <https://matplotlib.org/stable/tutorials/index>
- Seaborn is a library for making statistical graphics in Python, which builds on top of matplotlib and integrates closely with pandas data structures.
 - <https://seaborn.pydata.org/tutorial.html>

Types of Data Visualization

1. Univariate: Explore one variable (eg. Histogram, Boxplot)
 2. Bivariate: Explore relationships between two (eg. Scatterplot, Line plot)
 3. Multivariate: Explore multiple variables together (eg. Pairplot, Heatmap)
- Choosing the Right Plot
 - Check data distribution: Histogram / Boxplot
 - Compare categories: Barplot / Countplot
 - Identify relationship: Scatterplot / Lineplot
 - Find correlation: Heatmap
 - Detect clusters: Pairplot

Lab 2

- Study the world energy dataset (WorldEnergy.csv).
- Write a program Lab2.py in GitHub to perform EDA for the dataset and plot some meaningful graphs.
- In your report, explain what type of data analytics project (and the type of data analytics) can be done using this dataset.

(Note: Do not need to execute the project)

- You are encouraged to use generative AI to assist you on the coding only, but not the analysis.

- Lab Report 2 (with GitHubLink)